

Permissive Traffic signal control using Deep Reinforcement Learning

1. Introduction

Traditional traffic signal control methods utilise pre-programmed heuristics, with the intent of minimising congestion while ensuring safety. Traffic control is essential for transport planning, and these rules leave room for improvement. Advances in Artificial Intelligence have positioned Deep Reinforcement Learning (Deep RL) as an alternative to these rule-based methods, allowing the controller to learn more effective strategies.

This project aims to combine two existing methods within the Traffic-Control Reinforcement Learning community, with the aim of providing a structure capable of adapting by training to any single-intersection environment. Graph Neural Networks provide the architecture in which lane topology can be represented, utilising *flow* and *conflict* matrices to address spatial relationships between lanes. Furthermore, we can utilise Deep Q Network (DQN) Value Decomposition – allowing the agent to score each lane inside the intersection, effectively managing a combinatorial action space containing hundreds of unique phase configurations.

This report addresses the design, implementation, and evaluation metrics, using the SUMO environment wrapper. The objective was to develop an agent that could specifically focus on the complexities of permissive yielding / turning conflicts and adapt its choices accordingly. Experimental results demonstrate that the agent achieves superior throughput and individual vehicle waiting times when compared to heuristic fixed-time controllers.

2. Methodology

2.1. Simulation Environment

The chosen environment, Simulation of Urban MObility (SUMO) platform, aims to evaluate both the throughput and individual waiting time per vehicle. Unlike macroscopic simulators (e.g. CityFlow), SUMO simulates the kinematics and movements of each individual car, allowing a fine degree of control.

A challenge tackled was the problem of *permissive right-turns* (Left hand Traffic), in which vehicles would only enter an intersection when a “gap” in oncoming traffic forms. This gap-acceptance behaviour, coupled with shared-lane constraints, creates a critical dependency: a singular turning vehicle can obstruct the flow of all straight-moving traffic behind it. This introduces a stochastic element to the environment that heuristics often fail to consider – the throughput of permissive-turning vehicles is dependent on the density of higher-priority blocking traffic.

Multiple modifications have been made to the experimental environment to make the simulation more realistic. See Section 3.1 for more details.

2.2. State Representation

Each lane in the intersection is represented by a node on a graph. In a classical single-intersection scenario there are three types of lanes:

- *Input Lanes* refer to lanes in which vehicles “queue” before the intersection. Vehicles spawn in at the start of each input lane.
- *Internal Lanes* represent the specific trajectories inside the intersection itself. Scoring these lanes allows us to directly map to the status of each lane, allowing control over the direction each vehicle moves, emulating realistic traffic lights.
- *Output Lanes* are the lanes succeeding the intersection. Specifying the output lane capacity is essential for successful multi-intersection environments.

Standard Graph Neural Networks utilise only one adjacency matrix to represent the *edges*, or *connections* between nodes. However, to fully encapsulate the intersection topology within a GNN we need two adjacency matrices:

- *Flow Connections* A_{flow} represent the downstream movement of traffic. If a vehicle exits the end of one node, it must enter the start of another node, mapped through the flow matrix. Information about these neighbouring nodes helps the agent understand the relationships between nodes.
- *Conflict Connections* A_{conf} represent the physical conflicts between lanes of traffic. By nature, conflict connections only occur within internal lanes and allow the agent to understand which of these internal lanes conflict with others. Different types of physical conflicts (*diverging*, *merging*, and *T-bone*) are assigned a weight in accordance with their importance. The conflict matrix aims to represent the physical architecture of the intersection, allowing the agent to understand generic traffic rules without specifically hard coding them.

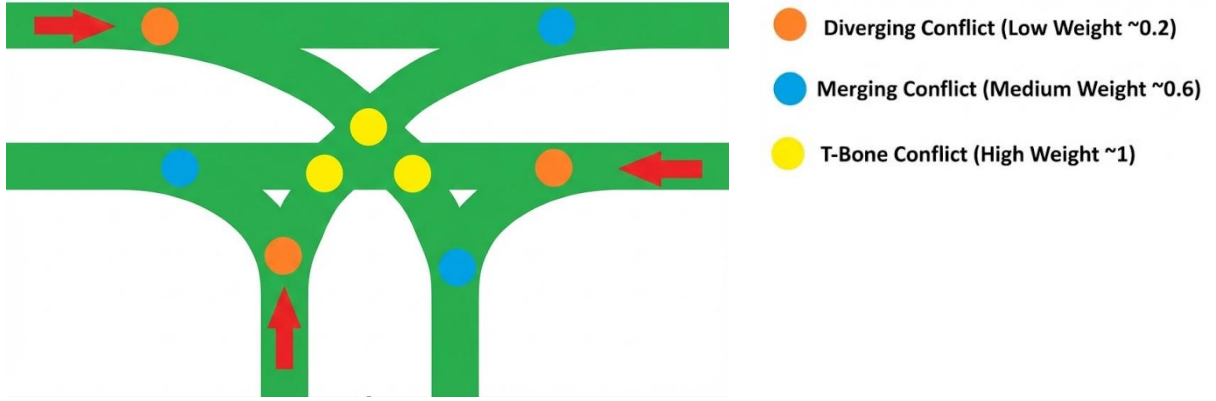


Figure 1: Taxonomy of Conflict Types in the Conflict Matrix for a regular T-Junction

The raw output from SUMO consists of variable-length lanes with continuous vehicle positions. Each node/lane is discretised into a 50-cell list, in which the physical distance between each cell is constant at 1 metre. This creates a fixed observation window optimised towards Deep Learning architectures.

In addition to this positional grid, we embed the directional intent of each vehicle into the embedding of the vehicle occupying each cell. This results in a matrix of size $X \in \mathbb{R}^{50 \times 3}$ where the second dimension represents a one-hot encoded vector representing the direction of each vehicle.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Figure 2: Example Node Vector Embedding of size $[18, 3]$. Vehicle positions are at index 0, 8, 11, 14, 17

2.3. Graph Neural Network Architecture & Value Decomposition

The Deep Learning architecture begins by processing each discretised lane-state using a shared, convolutional neural network. Through multiple convolutional and pooling layers, the vector embedding for each lane is compressed from $X \in \mathbb{R}^{50 \times 3}$ to a more manageable $h_1 \in \mathbb{R}^{64}$. A shared CNN is used because the purpose of this step is not to extrapolate relationships between lanes, but rather to better encode the information in each lane. Since this step is lane-invariant, we can therefore apply the same process to each.

Next, these latent vectors H are fed into the GNN, which performs the following message passing between layers:

$$M_{flow} = \sigma(A_{flow} * H * W_{flow})$$

$$M_{conf} = \sigma(A_{conf} * H * W_{conf})$$

W_{flow} , W_{conf} represent learned weight matrices implemented as linear layers, and A_{flow} , A_{conf} represent the intersection adjacency matrices. H , M_{flow} , and M_{conf} are then concatenated and compressed via a sequential block into a latent tensor of size $P \in \mathbb{R}^{26 \times 8}$, representing a vector of size 8 for each lane.

Finally, each lane's aggregated graph features are flattened, the *current phase vector* (green/red status based on the action before last) is concatenated to give the agent access to previous-step information $M_{flat} = \mathbb{R}^{208+14}$, and the flattened block is compressed into size $m = 14$, in which m represents the total number of internal (scoreable) lanes which represents our final Q values.

With these final Q values, the agent employs Value Decomposition to choose a traffic light phase. Unlike traditional approaches which estimate a single Q value out of a set of predetermined phases, this agent determines which phase to use from aggregating the scores of each lane.

For the set of all "safe" phases P as specified by Victorian Road Rules 2017, we find the phase P_j which results in the maximum possible value of $Q(s, P_j)$, determined by aggregating the Q-value of each active lane $\mathcal{L}(P_j)$ in said phase. This can also be represented through the following equation:

$$action = \arg \max_{P_j} \left(Q(s, P_j) = \sum_{l \in \mathcal{L}(P_j)} q_{lane}^{(l)} \right)$$

This allows the agent to understand the relationships between internal lanes through combinatorial generalisation, which methods that hardcode phase identification cannot. For example, this agent intuitively understands the relationship between phase $[N \rightarrow W, S \rightarrow E]$ and phase $[S \rightarrow N, S \rightarrow E]$, as it individually scores the lane $S \rightarrow E$ both times.

2.4. Training Details & Reward Function

To ensure stable training, the agent was trained using Double DQN with a Prioritised Experience Replay. In SUMO, each timestep represents a second for the purpose of realistic vehicle acceleration / maximum speeds. Key training hyperparameters include:

- Discount Factor: $\gamma = 0.9$. Relatively low because traffic lights under optimal conditions have the most impact within the first 25-50 timesteps.
- Exploration: Epsilon-Greedy Decay ranging from $1.0 \rightarrow 0.05$ with $\times 0.96$ decay per episode
- Target Update: Utilised Polyak Averaging with Tau ranging from $0.015 \rightarrow 0.001$ relative to epsilon
- Decision Frequency: every 25 timesteps

Training was divided into discrete Episodes rather than a continuous lifespan. Each episode consists of a 500-timestep simulation in which vehicles were spawned according to desired inflow. At the end of each episode, the agent is given up to 500×10 extra timesteps to flush remaining cars from the simulation before the episode is terminated. Episodic structure such as this prevented the agent from continuously overestimating the Q value for each step, leading to faster training convergence.

The agent operated through a meticulously crafted reward function with three major parameters:

- Throughput: How many vehicles were removed. Range of $[0, \infty]$
- Active Penalty: How many vehicles are currently active. Range of $[-\infty, 0]$
- Waiting Penalty: How long have each waited, power function. Range of $[-2, 0]$

$$r_t = w * \left(throughput_t - \alpha * active_{penalty}_t - \beta * wait_{penalty}_t^\epsilon \right)$$

Where $\alpha = 0.15$, $\beta = 0.0001$, $w = 0.2$ which aims to guide $\sigma(r)$ to 1 for faster value convergence, and $\epsilon = 1.5$ which penalises vehicles the more they wait at the intersection.

3. Experiments

3.1. Experimental Setup

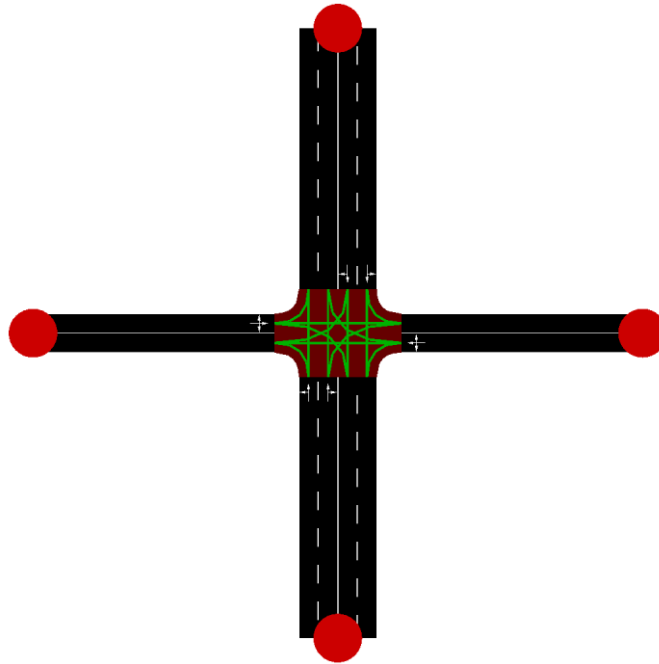


Figure 3: Example Intersection in which experiments are evaluated against. Input and Output lanes are represented by black lanes; Internal lanes are represented in the junction by green lines.

This simulation models a standard four-way intersection in which a two-lane arterial road intersects with a one-lane collector road. The arterial roads all share two directions, with the inner arterial, in particular, allowing both straight and right-turns to occur, creating situations in which lanes could be blocked. The collector roads, being only one lane wide, allow all three directions of travel again with right turns potentially being permissive.

This medium-density environment has been chosen, as it is proven that permissive turns are not suitable for high-density traffic, in which dedicated turning lanes are superior. In medium-density situations, however, permissive turning lanes often reduce the total amount of time.

To ensure the agent can generalise towards a variety of traffic conditions, two elements have been introduced into the environment

- **Vehicle Type:** Vehicles can exhibit different characteristics, including aggressive behaviour, sluggish behaviour, or the behaviour expected by trucks. Parameters such as vehicle size, speed, acceleration, and predictability have been modified within reason.
- **Inflow Rate:** Each episode is randomised with a variable throughput according to different traffic conditions. For an intersection such as this, the inflow rate is set between 2 and 8 per episode.

As part of the experimental setup a specific “decision frequency” hyperparameter was available – representing the time between actions. Ideally, the agent would be given access to the state every timestep and make an informed decision based on the given information. However, realistic intersections include an extended yellow light period and a minimum amount of time a light must be green for. Both features mean that on average, a specific traffic light phase should be active for around 20 to 30 seconds, depending on traffic queued, before being switched.

Initially, we aimed to create an agent that could intuitively understand these concepts and therefore make controlled decisions about when to switch certain lights to green/red more frequently, with the aim of encouraging the agent to *not* change phases. Before the agent makes a choice, the current phase of traffic lights is fed into the GNN allowing it a single-timestep memory. However, after testing, even through methods

such as reward shaping, the agent has the propensity to “flicker” between signals more rapidly than appropriate, meaning that a fixed decision frequency of every 25 timesteps was opted for instead.

3.2. Training Convergence

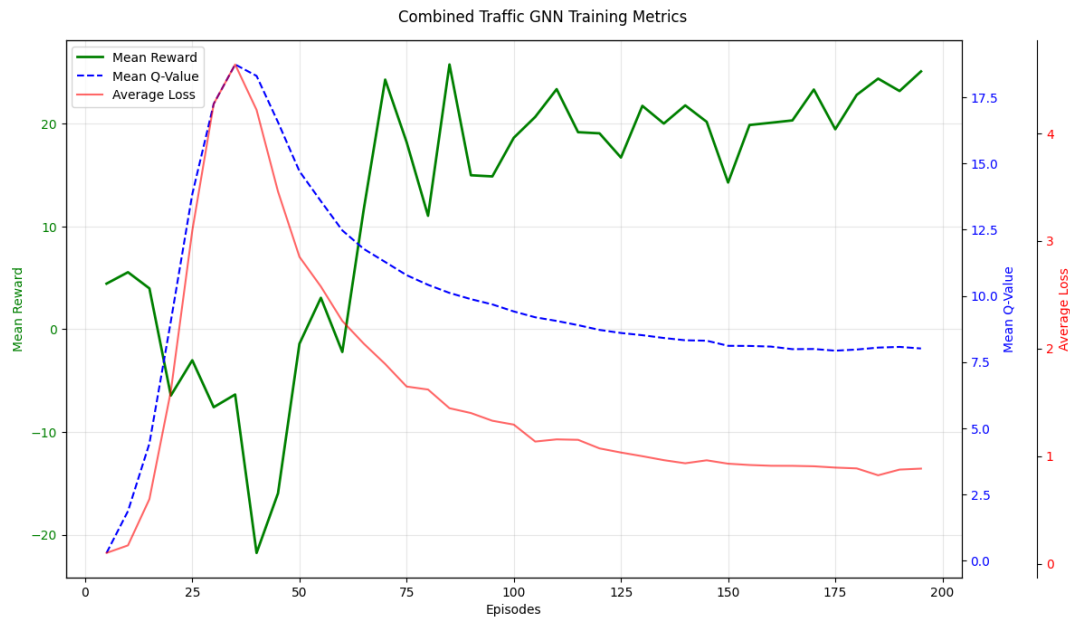


Figure 4: Relevant training metrics over 200 episodes.

Figure 4 reveals a three-phase training sequence. In the initial training phase between episodes 1 and 40, the agent exhibits Q-value overestimation. Under a high exploration rate $\epsilon > 0.25$, the agent incorrectly assumes that it can clear all vehicles without penalty, leading to inflated, representative value estimates. In the experiment shown the Mean Q-value rises to 18, which is higher than the corrected result but within a manageable range.

The second phase, between episodes 41 and 100, sees a consistent decline in Q values, followed with a sharp decline in TD Loss. A reduction in loss highlights that the target network is updating, and starting to realise that the policy network’s optimism and rise in Q values is unfounded.

Post episode 100, the agent enters a relatively stable equilibrium phase. The convergence of TD loss to an approximate value of 0.5 showcases that the agent has finished “correcting” its overestimation and has correspondingly exhibited a robust policy. During this phase, the agent makes micro-adjustments to its policy to squeeze marginal gains and maximise the reward.

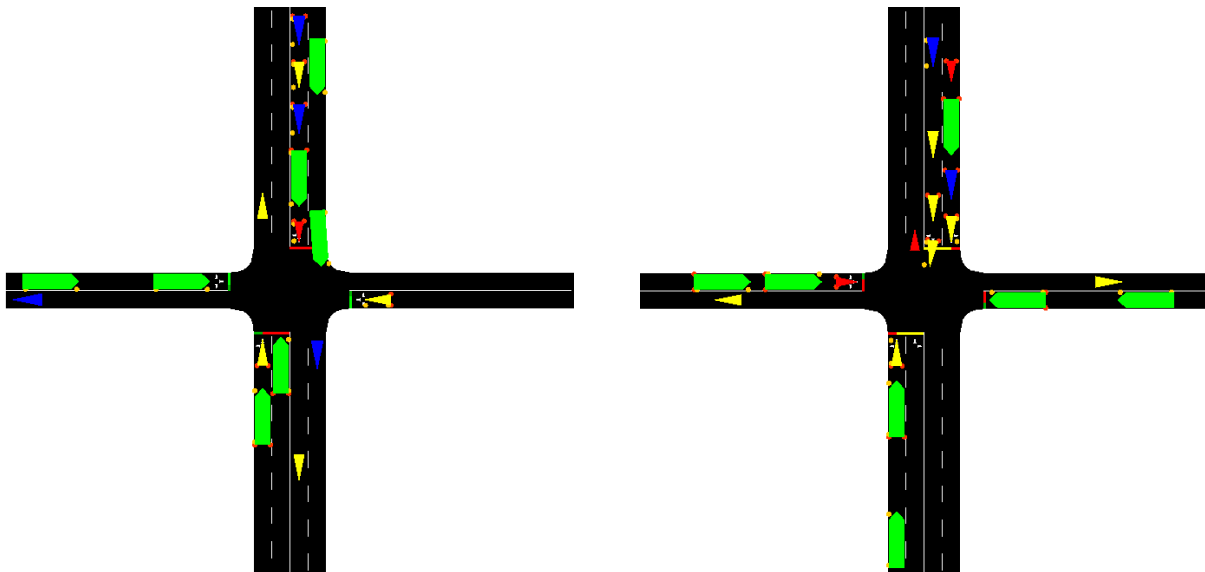


Figure 5: GUI representation, Agent performing signal changes in the experimental intersection with traffic

3.3. Baseline Comparison

To evaluate the effectiveness of this agent, its performance was benchmarked against a rule-based greedy heuristic. In essence, this heuristic chooses the phase action the same way that the agent does – through the argmax of internal lanes. However, instead of *learning* appropriate lane-values like the agent, each internal lane’s score is entirely dependent on the number of queueing vehicles aiming to use this internal lane. This method shares fundamentals with Max Pressure control theory, as both rely on queue lengths to determine priority. However, while Max Pressure utilises the differences between both upstream and downstream connections, our experiment assumes infinite output capacity, simplifying the problem to only upstream demand.

Two experimental runs were individually conducted to compare the baseline to the GNN model (Figure 6):

Experiment 1				Experiment 2			
Metric	Base	GNN	% Diff	Metric	Base	GNN	% Diff
Avg. Wait time (s)	14.2	14.62	+3.1	Avg. Wait time (s)	15.8	16.5	+4.6
Max Wait time (s)	90.2	75.0	-16.7	Max Wait time (s)	76.1	86.9	+14.5
Avg. Queue length	10.0	8.4	-16.0	Avg. Queue length	9.8	9.2	-6.1
Avg. Speed (km/h)	8.2	8.7	+5.5	Avg. Speed (km/h)	8.8	9.3	+5.3
Avg. Time loss (s)	19.4	20.7	+6.7	Avg. Time loss (s)	22.9	22.0	-4.0
Throughput (v/h)	1857	1814	-2.3	Throughput (v/h)	1958	1792	-8.5
Total Reward	33.5	33.75	+0.7	Total Reward	34.52	31.33	-9.2

Figure 6: Experimental results for GNN vs baseline

The data indicated that the agent successfully converges to the optimal greedy strategy represented by the baseline. Its failure to outperform the baseline can largely be attributed to its inability to learn new concepts about increasing traffic flow without major throughput repercussions to its existing strategy. To improve, the agent needs to be able to attribute negative scores to each internal lane, effectively blocking them from phase selection. An example where this is appropriate is regarding dedicated right-turning lanes, which are currently always superseded by permissive right turning lanes.

One reason that this occurred is due to the shallow state representation – currently, only position and direction values are encoded into a state. While this provides enough information to construct a greedy solution, the agent might need access to more data, such as current wait time, velocity, and vehicle type, to learn a more valuable policy.

The reward function contains a penalty based on the power of how long each vehicle has been waiting for. This means that vehicles that have been waiting longer have a penalty per timestep to the reward, penalising the agent according to realistic concerns. Lack of any meaningful state memory storage, or information outside of position + direction, means that the agent physically cannot prioritise vehicles based on their existing wait time.

4. Conclusion

In conclusion, this project demonstrates a Graph Neural Network-based reinforcement learning approach that can learn to control a complex traffic intersection from scratch. The training environment was built using the SUMO simulator – utilising a custom wrapper and able to control the movements of individual cars, allowing a fine degree of control and high simulation accuracy. The agent used a Graph Neural Network to capture spatial dependencies that allow the efficient processing of traffic flow. We utilise flow and conflict adjacency matrices to give the agent information about the intersection, and value decomposition to aggregate individual lane scores into a large set of actionable phases.

Experimental results yielded an often large, but controlled, rise of Q values during the exploration phase, followed by a steady decrease and convergence during training. When compared to a greedy rule-based heuristic following the same value-decomposition methods as our agent, the GNN model achieved

performance parity, functionally learning a simple queue-demand policy. In the future, the aim is to fine-tune this model, further allowing it to capture long-term details and become more effective than the baseline.

5. References

- [1] Chang, A., Ji, Y., Wang, C., & Bie, Y. (2024). CVDMARL: A Communication-Enhanced Value Decomposition Multi-Agent Reinforcement Learning Traffic Signal Control Method. *Sustainability*, 16(5), 2160. <https://doi.org/10.3390/su16052160>
- [2] Hasselt, van, Guez, A., & Silver, D. (2015). *Deep Reinforcement Learning with Double Q-learning*. ArXiv.org. <https://arxiv.org/abs/1509.06461>
- [3] Levin, M. W. (2023). Max-Pressure Traffic Signal Timing: A Summary of Methodological and Experimental Results. *Journal of Transportation Engineering, Part A: Systems*, 149(4). <https://doi.org/10.1061/jtepbs.teeng-7578>
- [4] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). *Prioritized Experience Replay*. ArXiv.org. <https://arxiv.org/abs/1511.05952>
- [5] *Traffic Lights - SUMO Documentation*. (n.d.). Sumo.dlr.de. https://sumo.dlr.de/docs/Simulation/Traffic_Lights.html
- [6] Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019). PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. <https://doi.org/10.1145/3292500.3330949>
- [7] Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., & Li, Z. (2019). CoLight: Learning Network-level Cooperation for Traffic Signal Control. *28th ACM International Conference on Information and Knowledge Management*. <https://doi.org/10.1145/3357384.3357902>